

Пайплайны данных рекомендательных систем

ШАД, Рекомендательные системы
2025/26, лекция 8

Даня Ткаченко

Руководитель службы ML-сервисов,
Яндекс Лавка

Яндекс



Мы уже умеем строить модели

- Семь лекций мы разбирали как рексистемы работают изнутри: кандидатогенерация, ранжирование, лоссы, архитектуры, bias'ы.
- Всё это невозможно без данных
- В большой системе работа с этими данными становится сложной инженерной задачей
- Сегодня мы посмотрим на основы вращения данных в вашем продакшене.



План занятия

На лекции:

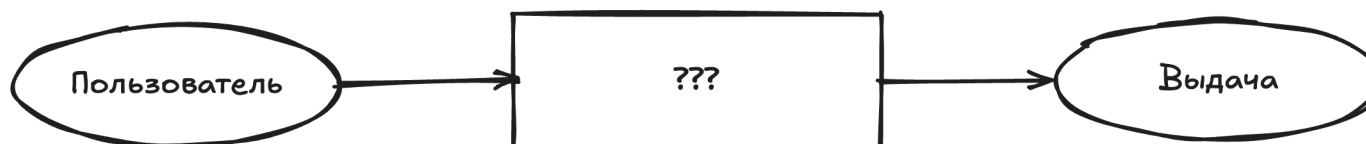
1. Архитектуры хранения и обработки
2. Логгирование и обучение
3. Качество данных и дрифт

На семинаре:

- Поподробнее обсудим feature store'ы

Попробуем построить сервис рекомендаций

- Будем поддерживать схему работы нашей системы
- Постепенно дорисуем ее, пока не получим что-то похожее на приличную production-архитектуру.



Архитектуры хранения и обработки данных

Часть 1



Живем в двух таймлайнах

Что нужно модели в момент запроса:

- Пользователь любит молочку — молочку надо показать выше
- Молоко уже в корзине — больше не надо показывать
- Сейчас вечер пятницы — показываем пиво/чипсы/сухарики побольше

Что это значит технически:

- «Любит молочку» — агрегат за 90 дней, на запросе за 50ms особо не посчитаешь
- «Молоко в корзине, вечер пятницы» — можно увидеть только в реальном времени

Живем в двух таймлайнах

Что нужно модели в момент запроса:

- Пользователь любит молочку — молочку надо показать выше
- Молоко уже в корзине — больше не надо показывать
- Сейчас вечер пятницы — показываем пиво/чипсы/сухарики побольше

Что это значит технически:

- «Любит молочку» — агрегат за 90 дней, на запросе за 50ms особо не посчитаешь
- «Молоко в корзине, вечер пятницы» — можно увидеть только в реальном времени

- Разные данные имеют разную свежесть и разную стоимость подсчета.
- Нельзя всё считать в рантайме — дорого.
- Нельзя всё считать заранее — неактуально или просто невозможно.

Офлайн и онлайн

Базовый вариант этого разделения:

Офлайн:

- Эмбеды айтемов
- Пользовательские статистики

Обновляем условно раз в сутки или по триггерам

Онлайн:

- Последние 5 кликов в сессии
- Геолокация, устройство, время суток

Должны считать потоков или получать в запросе

Часто граница разделения не концептуальная, а скорее инженерная.

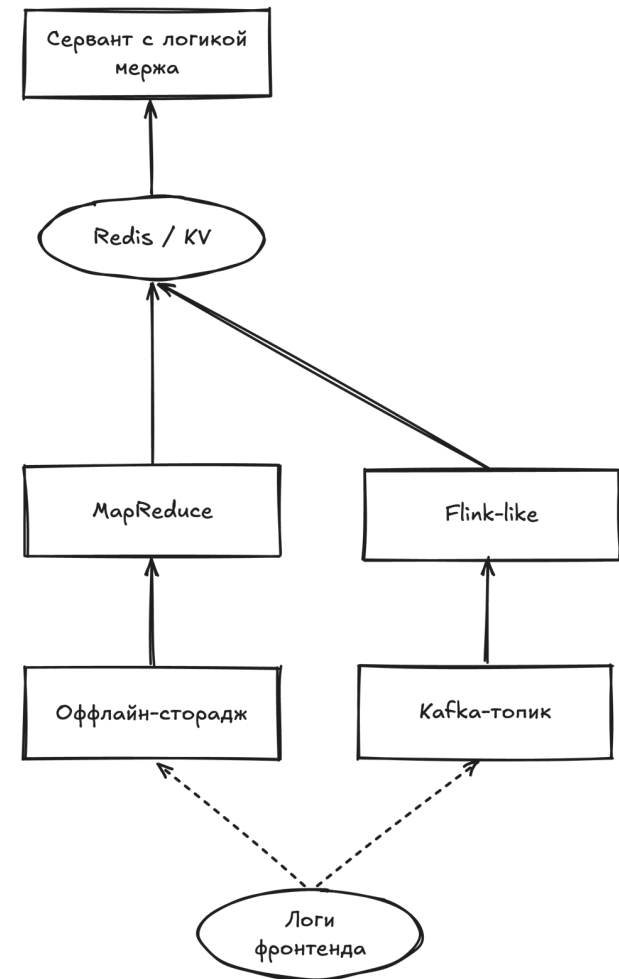
- Одну и ту же фицу можно реализовать там или тут, но будет стоить разных денег и давать разную свежесть
- Сами границы оффлайн и онлайн могут быть размыты, если мы часто пересчитываем в оффлайне большие батчи

На практике: лямбда-архитектура

В реальности же нам нужно и долгое (месяцы), и свежее (часы).

Три слоя:

- **Batch Layer** (Spark / YTSaurus) — читает полную историю, пересчитывает раз в сутки
- **Speed Layer** (Kafka + Flink / LB + BigRT) — обрабатывает поток событий из рантайма с задержкой в секунды
- **Serving Layer** (Redis / Ваш любимый KV Storage) — объединяет результаты обоих слоёв

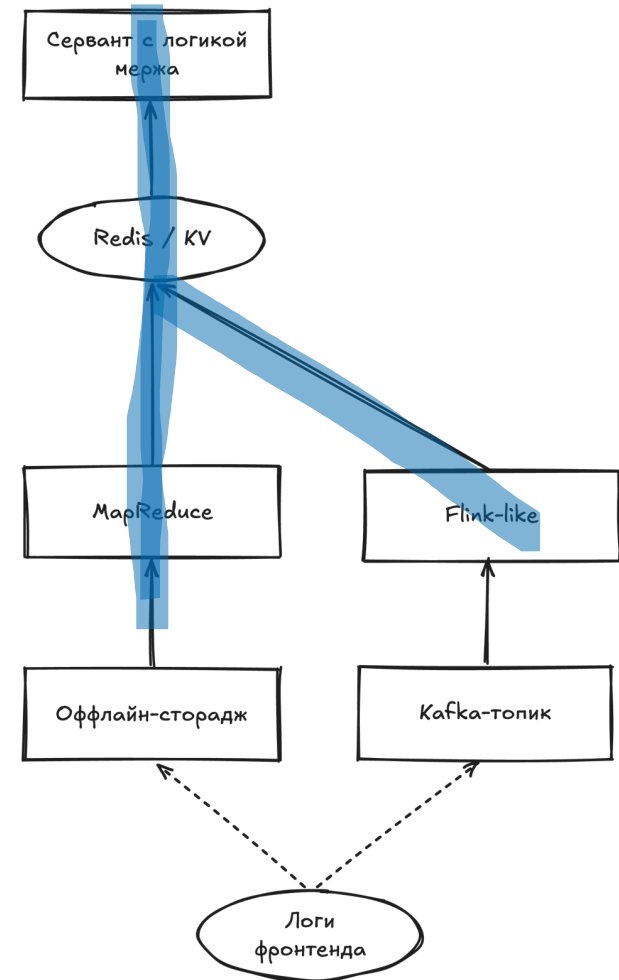


На практике: лямбда-архитектура

В реальности же нам нужно и долгое (месяцы), и свежее (часы).

Три слоя:

- **Batch Layer** (Spark / YTSaurus) — читает полную историю, пересчитывает раз в сутки
- **Speed Layer** (Kafka + Flink / LB + BigRT) — обрабатывает поток событий из рантайма с задержкой в секунды
- **Serving Layer** (Redis / Ваш любимый KV Storage) — объединяет результаты обоих слоёв



Batch layer

- Периодически (раз в час/день) тяжёлый mr-job читает полную историю
- Пересчитывает агрегаты с нуля
- Пишет результат в serving layer



Пример из Лавки:

Долгосрочный профиль пользователя:

- топ-категории за 90 дней
- средний чек
- частота заказов
- тяжелые эмбединги по длинной истории

Считаем раз в сутки, к утру в кв лежат статистики с учетом прошлого дня.

Speed layer



Считает за какое-то не очень большое время статистики по стриминговым событиям (чаще всего до пары суток), чтобы потом можно было приклеить это к оффлайн-части данных

Значительно дороже по железу и сложен в девопсе.

Дает жизнь главной боли лямбда-архитектур: одна и та же логика агрегации живёт в двух местах — batch и speed.

- Баг нужно чинить дважды.
- При изменении логики деплоить нужно тоже дважды.
- Batch на Spark, speed на Flink — разные языки, часто даже разные команды.
- Что важно для нас: легко разъехаться по данным

Решение – каппа-архитектура?

- Если наши стриминговые системы инженерно достаточно развиты (а разработчики очень смелые) – уберём batch совсем.
- Всё считаем в потоке, для исторического пересчёта запустим тот же код на эмуляции Kafka-топика по старым данным
- Получится одна кодовая база.

Проблемы:

- Все равно не сможем обрабатывать столько данных, сколько смогли бы с MapReduce, такая обработка дороже по железу
- Добавить новую фичу становится нехилым инженерным приключением почти всегда (а не только, когда она свежая)
- Нет красивого объяснения у названия



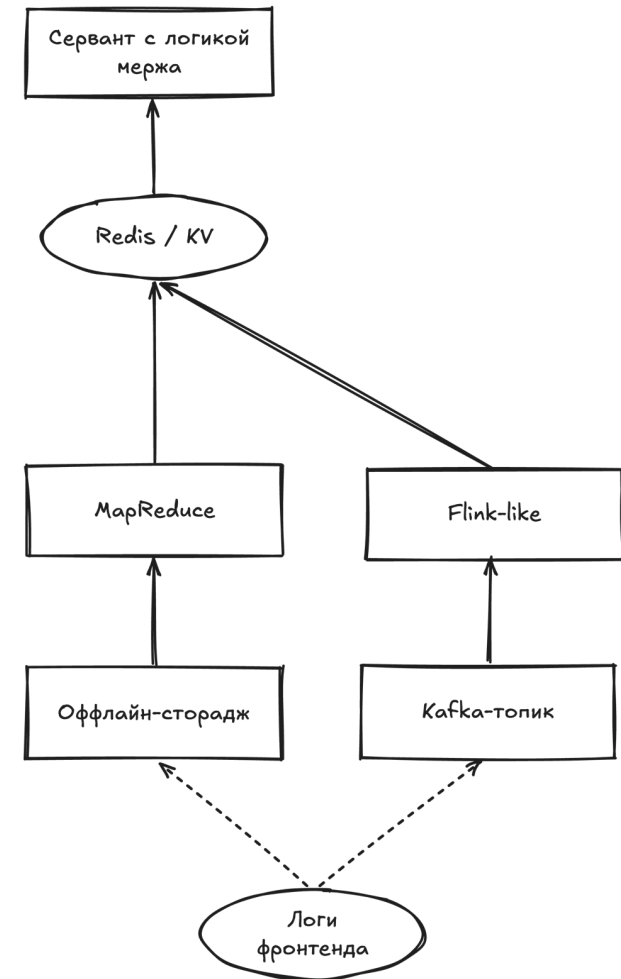
Важный момент

Для получения данных мы чаще всего не ходим в два (или один, зависит от нашего решения) KV-storage'a напрямую.

- Для логики управления мержем обычно вклинивают отдельный сервис.
- В случае, если мы говорим о пайплайне для пользовательских статистик, его обычно называют как-то в духе User Profile Service.

Что он делает:

- Получает user_id / cookie_id
- Читает из KV-зеркал всё, что знаем о пользователе
- Делает хитрый мерж, чтобы не задублировать данные
- Поддерживает фоллбеки на недоступность
- Возвращает сериализованную информацию -- юзерский профиль



Пример всего вместе

Фича: количество покупок молочки у пользователя за 30 дней

Batch (раз в сутки):
Spark читает все заказы за 30 дней
→ считает количество
→ пишет в Redis.

Speed (на каждое событие):
Новый заказ в Kafka
→ Flink инкрементально обновляет счётчик
→ пишет delta в Redis.

*Инкремент для скользящего окна нетривиален — нужно не только прибавить новое, но и вычесть старое. Обычно хранят кольцевой буфер событий.
Если хотим стейтлесс, можно использовать экспоненциальное угасание по времени.*

UPS

- Забирает профиль из Batch-части Redis'a, смотрит на то, когда она обновлена
- Добирает нужное количество дельт из Speed-части Redis'a

Видите проблему?

Что в большинстве рантаймов?

В большинстве своем какой-то вид лямбды (мб менее академический, чем на картинках выше)

Как тогда бороться с двумя имплементациями для статистик?

- Условная Spark-дジョба (Python/Scala) считает возраст клиента как
`(now - first_order_date).days`
- Флинк (Go/Java) считает то же как
`(now - registration_date).days`
- Не со зла получились разные числа

Ответ – какой-то вид feature store

Другой пример:

- Офлайн-код пишет одна команда, онлайн – другая.
- Договорились сделать одинаково, но со временем немного разъехались.
- Модель обучается на одном, инферит на другом

Feature Store



Feature Store – это общее название для всех абстракций (и их имплементаций), которые используются, чтобы гарантировать одинаковый способ подсчета фичи между всеми клиентами.

- Чаще всего некоторый вид лямбда-архитектурного процессинга с общей библиотекой обработки фичей
- Одна реализация логики — и офлайн, и онлайн
- Разные плюшки: версионирование, инвалидация и прочее
- Можно использовать сразу и для обучения, если наша офлайн-часть гарантирует point-in-time корректность тем или иным образом

Больше про это на семинаре

Типичная ситуация, если забыть на это

Выкатили эксперимент, что видим:

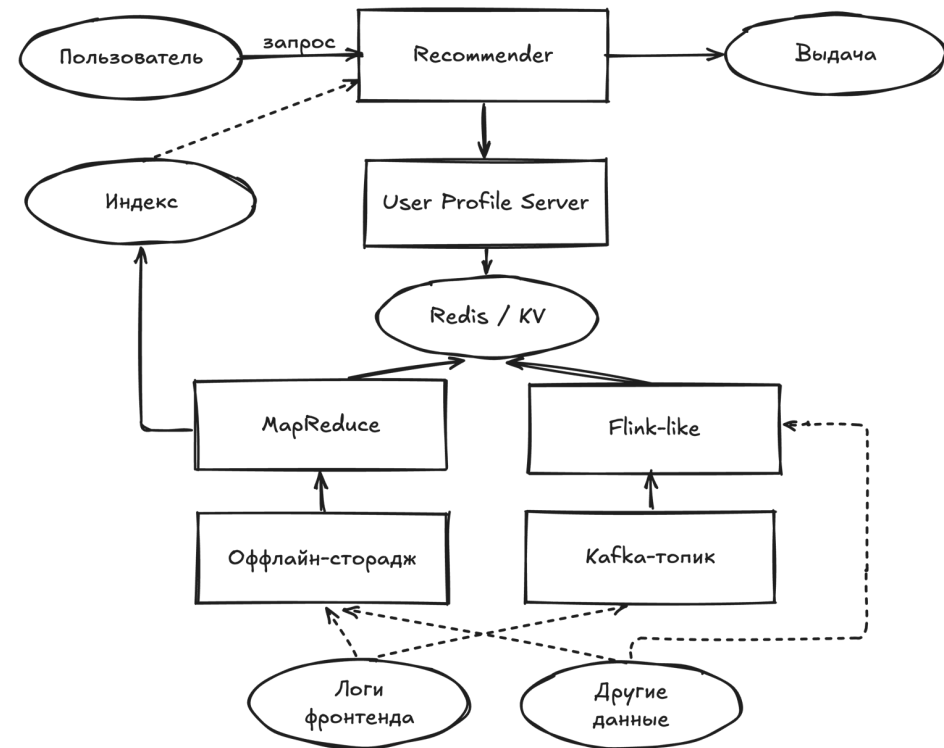
- Офлайн-метрики хорошие, модель на валидации выглядит отлично
- В A/B тесте онлайн-метрики хуже — непонятно почему
- Долго дебажим случайные вещи
- В итоге мучительно сравниваем значения фич офлайн против онлайн и видим проблему

Feature skew — вторая (после неправильного первоначального выбора) причин расхождения офлайн и онлайн метрик. Feature Store помогает решать эту проблему



Посмотрим на всю картинку:

- Запрос прилетает в сервис скоринга (ака Recommender)
- Recommender получает профиль пользователя из UPS
- На нем уже откуда-то есть индекс для айтемов
- В самом запросе была какая-то контекстная информация
- Теперь уже запускаем всю рексистему с кандидатами и финальной моделью
- Отдаем выдачу



Экзотический вариант, если нам нужны **очень** тяжёлые модели

Обычно у нас есть до 100-200 миллисекунд на ответ. Что делать, если мы хотим инференсить модель, которая инференсится дольше (потому что мы невероятно заскейлили по качеству), при этом используя свежие данные?

Можно попробовать разные виды NearRT конструкций, базовая идея:

- вычисляем скоры моделей асинхронно по каким-то триггерам
- результат кладём в быстрое на чтение хранилище
- в реквест-тайме recommender просто сходит туда за скорями

Экзотический вариант, если нам нужны **очень** тяжёлые модели

Варианты триггеров:

- По превышению степени двойки какого-то счетчика (число просмотренных секунд)
- По ключевым событиям (добавление в корзину)
- По числу событий (обычно тоже что-то со степенями двойки)
- По времени (раз в 10 минут)

Это lowkey экзотика, нужно только когда хотим катать прямо **очень** тяжёлые модели.

Логгирование и обучение

Часть 2



Что нужно для обучения

Для обучения нам нужно восстановить состояние системы в момент показа:

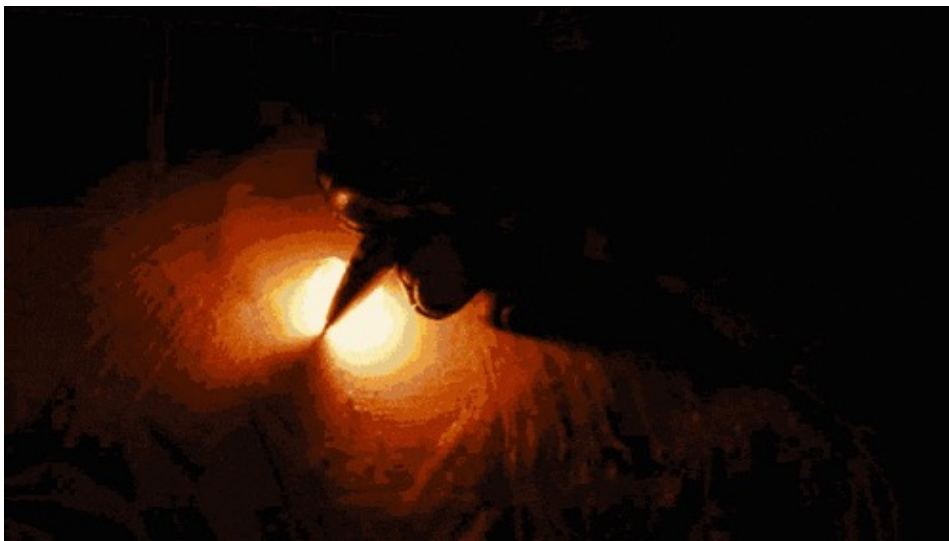
- Какие айтемы показали пользователю
- Какие фичи видела модель в момент, когда строила это ранжирование
- Что пользователь сделал (таргет)

Проблема:

Фичи пользователя постоянно меняются. Как восстановить их значения на момент конкретного показа?

Посмотрим на два противоположных подхода.

Подход 1: залогированные фичи



В момент инференса на Recommender пишем в лог значения фич, которые использовались для предсказания.

На клиенте пишем логи взаимодействий.
Джойним это дело.

Плюсы:

- Точно то, что видела модель
- Нет рисков по разному распределению фичей
- Можно воспроизвести предикт в оффлайне

Минусы:

- Дорого по месту
- Сложно добавлять новую фичу, нужно ждать, пока залоггируется

Подход 2: реконструкция фичей



Не логируем фичей, логируем только клиентскую часть. Значения фичей как-то восстанавливаем в оффлайне.

Плюсы:

- Мгновенно добавляем новые фичи
- Не надо тратиться на место под логи Recommender'a

Минусы:

- Восстановить точно примерно невозможно
- Не все фичи вообще можно восстановить, ограничивает дизайн
- При особой кривоте рук можно получить и лики

Подход 2: реконструкция фичей



NB: при использовании Feature Store с Point-in-time correctness вопрос в целом снимается. При дизайне такого Feature Store нам неизбежно придется принимать решение о том, какие фичи мы логируем, какие фичи рассчитываем заново в оффлайне, а какие берем из логов запросов.

Не логируем фичей, логируем только клиентскую часть. Значения фичей как-то восстанавливаем в оффлайне.

Плюсы:

- Мгновенно добавляем новые фичи
- Не надо тратиться на место под логи Recommender'a

Минусы:

- Восстановить точно примерно невозможно
- Не все фичи вообще можно восстановить, ограничивает дизайн
- При особой кривоте рук можно получить и лики

Минимальный необходимый набор логов

Бэкендные логи (Recommender):

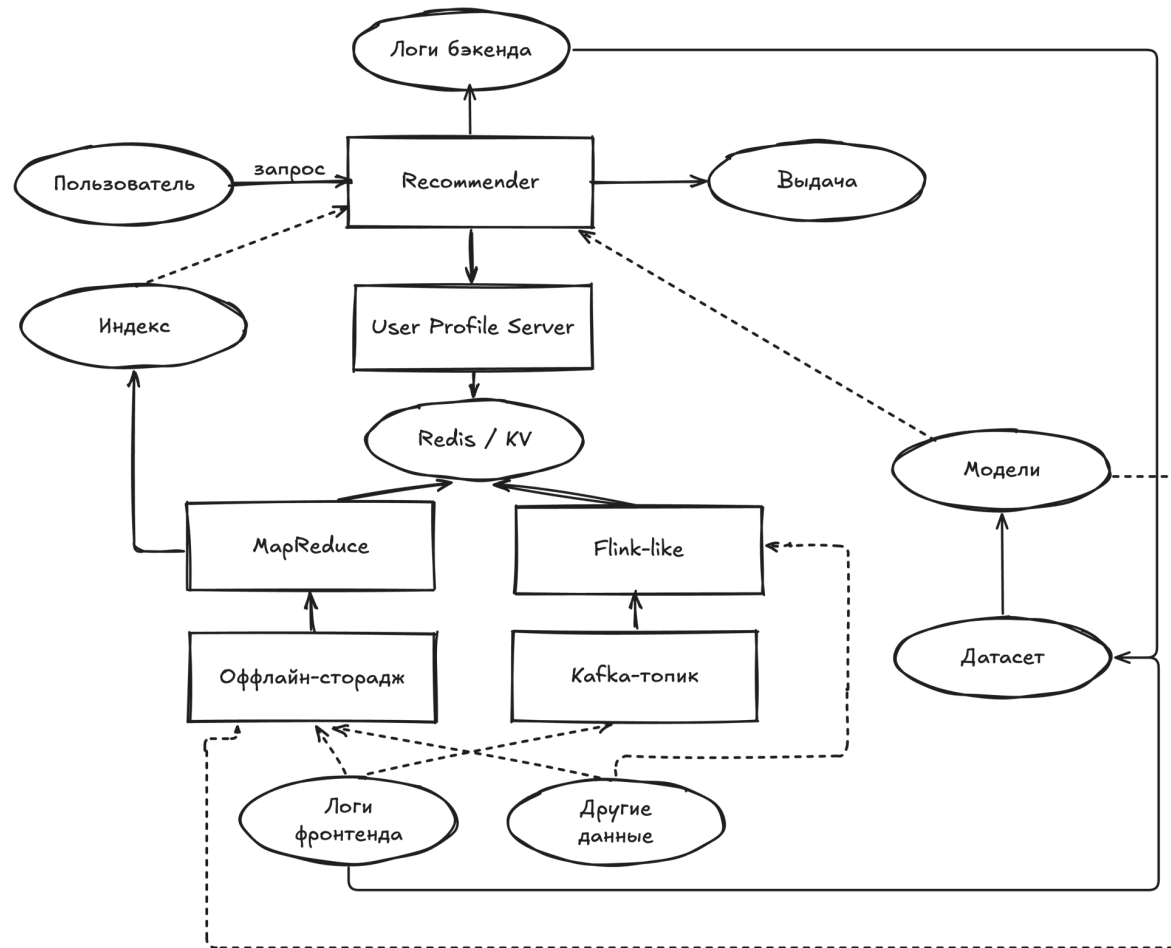
- id запроса, пользователя, айтема
- серверное время
- позиция
- скор модели

Фронтендные логи:

- очень важно: id запроса, с которого пришел айтем
- id айтема, пользователя
- время тут нужно использовать серверное по падению лога в хранилище

Оба потока мержим по request_id – это единственный честный способ получить настоящую картину произошедшего. При прочих равных отдаем предпочтение бэкендным логам.

Дорисуем нашу схему



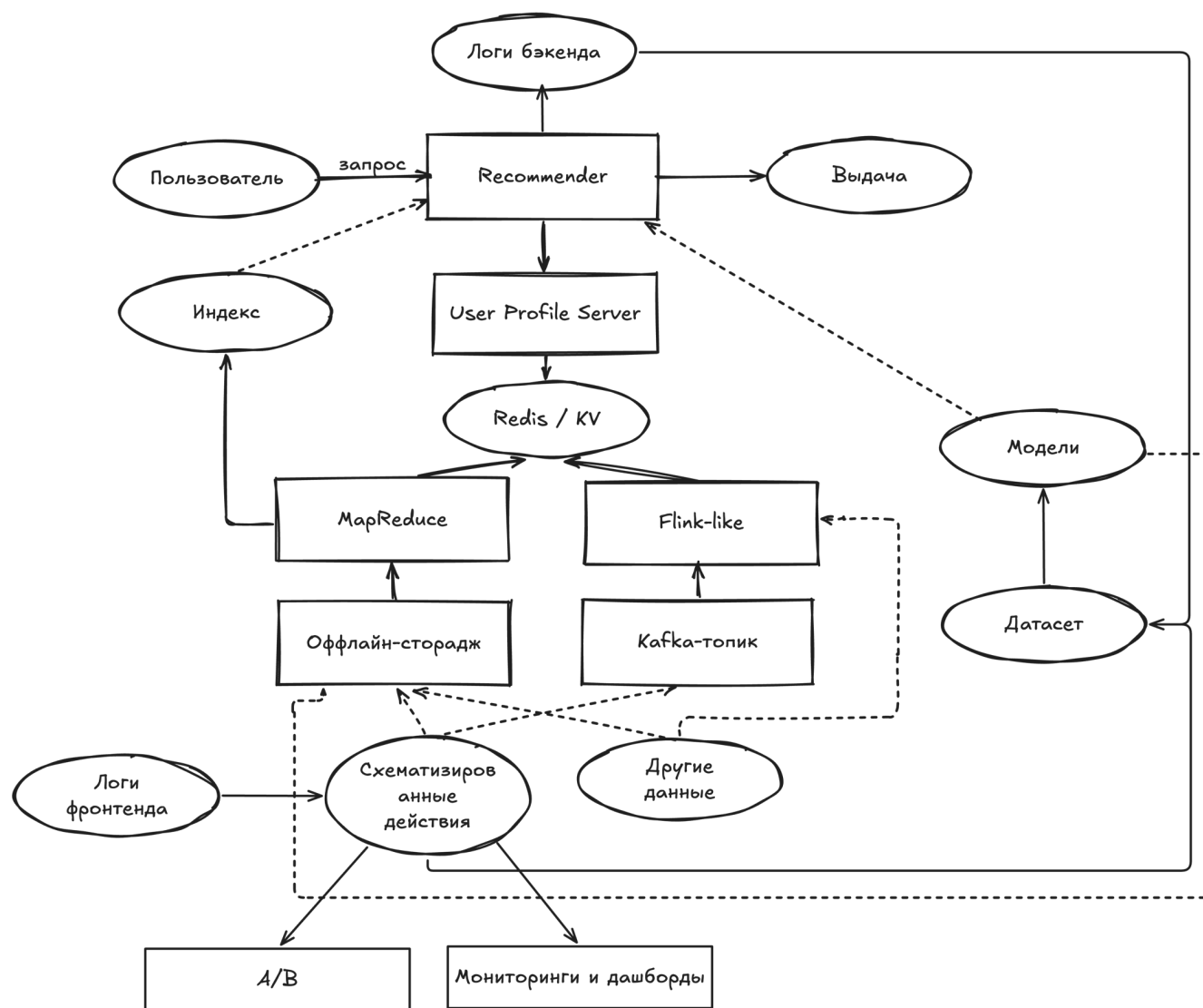
Практический момент

Одно фронтное событие в последствии используется в

- Построение датасета для модели
- Для подсчета фичей
- В A/B для подсчета метрик эксперимента
- В мониторингах и дашбордах

Помимо этого фронтные логи могут прожить большую жизнь и пройти через поколения разработки, что не помогает с их парсингом.

Вывод: нужна единая схема для пользовательского события.



Продолжаем строить обучающую выборку

Есть залогированные данные, чтобы построить пул нужно:

- Смержить показы с логами по request_id
- Например, посабсэмплить по пользователям

Можем столкнуться с тем, что один и тот же реальный человек может иметь несколько разных идентификаторов в системе. История пользователя оказывается разорвана между ними.

Один пользователь — много идентификаторов

История пользователя разорвана между разными id:

- user_id: 42 (когда залогинен)
- device_id: iphone_X
- device_id: macbook_Y
- cookie_id: abc123

Последствия

- Персонализация хуже чем могла бы быть.
- Наблюдаем что-то типа фейкового холодного старта

Где это особенно критично

Кажется, что потерять небольшой кусочек истории может быть не очень страшно, но в некоторых рексистемах это важно.

Пример: adult-контент платформы

Пользователи осознанно скрывают поведение:

- Смотрят через incognito — новый cookie_id каждый раз
- Не логинятся — нет user_id

Если не матчить айдишники — у каждого пользователя будут холодные реки.



Cross-device matching здесь напрямую влияет на качество рекомендаций и retention. Это мотивирует инвестировать в ID Graph.

Более понятный кейс: залогин

Сценарий:

- Неделю ходил без залога, накопил историю
- Решил зарегистрироваться, теперь есть user_id

Отсюда вывод, что важно работать с правильными айди на уровне Feature Store

Качество данных и дрифт

Часть 3



Что мы уже видели

Проблемы которые мы уже видели:

- Feature skew: офлайн и онлайн расходятся
- Немного про потери клиентских событий
- Data leakage при реконструкции фич
- Потеря истории пользователя из-за множественных айдишников

Теперь поговорим о том, как за этим следить.

Метрики модели

Базовый минимум:

Посчитать ваши оффлайн-метрики (e.g. AUC/NDCG)
на слепке пула за окно в последние X дней.

Роскошный максимум:

- Метрики отдельно по дням/часам — видны аномалии и тренды
- Разбивка по когортам: новые vs старые пользователи
- Разбивка по платформам: web vs mobile vs app
- Сезонность: будни vs выходные

Метрики качества данных: покрытие фич

Покрытие фичи — доля запросов для которых фича доступна:

- user_age_days: 99.8%
- user_category_affinity: 87%
- item_embedding: 94%
- user_geo_cluster: 71%

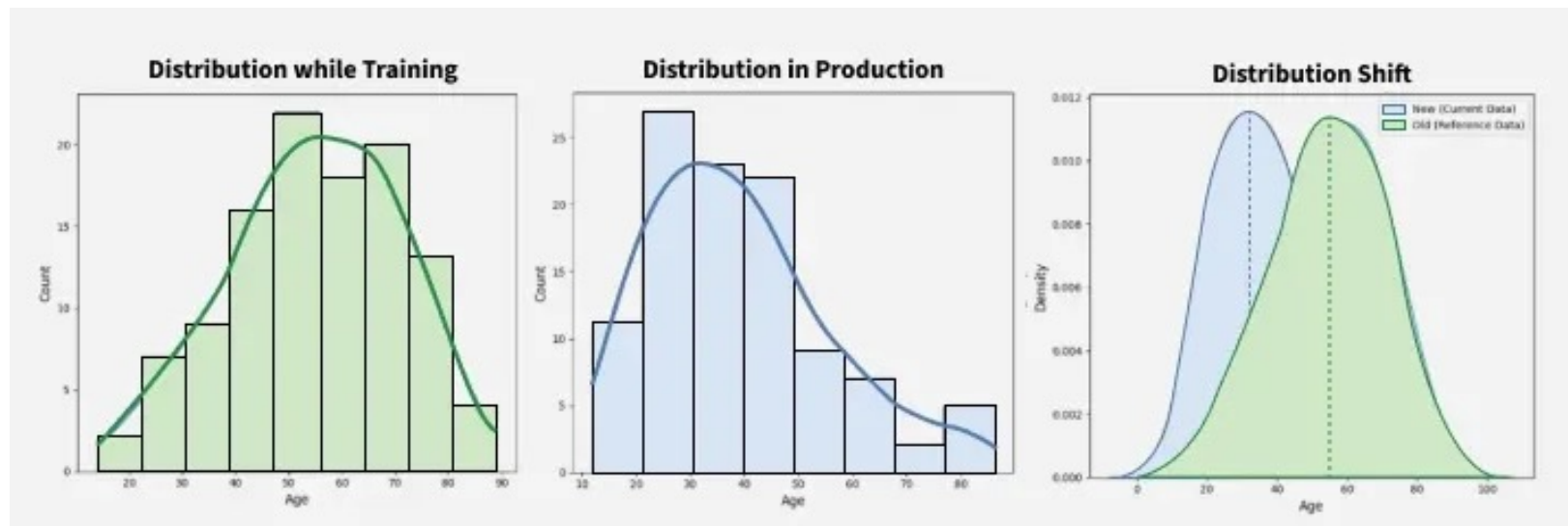
Модель обучается на выборке с 87% покрытием, в проде фича может быть доступна для 60% запросов. Поведение на примерах без фичи непредсказуемо – в индустрии в таких случаях говорят, что модель «едет».

Резкое падение покрытия — почти всегда upstream-проблема (сломался пайплайн, изменилось API). Нужно ловить до того как отразится на метриках.

Метрики качества данных: аномалии в распределениях

Что мониторить:

- Резкий сдвиг статистик распределения
 - среднего / медианы / квантилей фичи
- То же самое в когортных разрезах
- Производные этих статистик



Data drift: когда модель едет

Общий термин — data drift: распределение данных изменилось.
Три разновидности:

Covariate drift — изменилось распределение входных фич X :

Пришла новая когорта пользователей, запустилась рекламная кампания, сменился сезон.

Label drift — изменилось распределение таргета Y :

Кликрейт вырос потому что пришла более лояльная аудитория, а не потому что модель лучше.

Concept drift — изменилось само отображение $X \rightarrow Y$:

То что раньше нравилось пользователям — теперь нет. Тренд устарел.

Детекция: практические инструменты

Для фич (covariate drift):

- PSI (Population Stability Index)

$$PSI = \sum \left((Actual\% - Expected\%) \times \ln \left(\frac{Actual\%}{Expected\%} \right) \right)$$

- Предыдущий слайд

Для предикта модели:

- Распределение скоров на инференсе — среднее, std, перцентили

Для таргетов (label drift):

- Посуточный CTR, conversion rate по сегментам

Нашли дрифт — что дальше?

Covariate drift:

Обычно помогает переобучение на свежих данных. Чинимся и идем восстанавливать upstream.

Label drift:

Разбираемся в причине:

- сезонность (ждём/переобучаемся)
- изменение продукта (пересматриваем таргет)
- баг в логгировании (чиним срочно).

Concept drift:

Самое малопонятное. Переобучение помогает, только если сам контент и/или продукт успевает за изменениями. Иногда нужно пересматривать постановку задачи.

Пример: перед праздниками паттерны меняются кардинально и предсказуемо. Готовимся заранее: праздничная модель или апсемплируем соответствующий период прошлого года.

Конвейеры обучения: варианты

Ручное переобучение:

Обучаем когда хотим, деплоим руками.

Хорошо для старта, плохо в сложном многочастном продакшене.

Переобучение раз в период:

Конвейер по расписанию (daily/weekly).

Переобучение по триггеру:

Переобучение по событию — метрика упала, задетектирован дрифт, накопилось N новых примеров. Что-то на богатом.

Для большинства задач достаточно
дневного/недельного переобучения. Триггерное
нужно там где данные меняются очень быстро —
новостные рекомендации, тикетная торговля.

Почему у нейронных моделей тут удобнее

Бустинг:

- Обучается с нуля каждый раз
- Нельзя дообучать маленькими батчами

Нейронные сети:

- Fine-tuning: дообучаем поверх старых весов, не с нуля
- Можно заморозить нижние слои, дообучать только голову
- Эмбединги новых айтемов обновляем инкрементально

Онлайн-дообучение: риски

Нейронка может забывать паттерны, которые были важны месяц назад.
Критично для редких сегментов пользователей.

Feedback loop усилится:

 модель рекомендует →

 пользователи смотрят рекомендованное →

 данные смещены →

 модель усиливает смещение.

Онлайн-дообучение ускоряет это дело.

Аномалии: бот-атака или сбой → модель быстро «впитывает» аномалию.

Обычно используем какую-то комбинацию

Что мы узнали сегодня

- Лямбда = batch + speed layer — гибрид для полной истории и свежести одновременно
- Feature Store — одна логика вычисления фичи для офлайна и онлайн
- Два подхода к обучению: логируем фичи или реконструируем
- Нужен единый способ использования фронтовых логов
- ID Graph — один пользователь может выглядеть как несколько
- Data drift и как за ним смотреть
- Про конвейеры переобучения



tg: @deadpadre

Спасибо!

Даня Ткаченко,
Руководитель службы ML-сервисов Яндекс Лавки

Белград 2026

